

Traditional Image Segmentation Algorithms: An Empirical Comparative Evaluation

Prepared By:

Wasi Al Safa
Roll: 2103058, Series: 21

Sunan Rabby
Roll: 2103037, Series: 21

Md. Shibbeer Thalha
Roll: 2103055, Series: 21

Submitted To:

Utsha Das
Assistant Professor

Department of CSE
Rajshahi University of

Engineering & Technology

Department of Computer Science and Engineering (CSE)
Rajshahi University of Engineering & Technology

July 6, 2026

Contents

1	Introduction and Project Objectives	2
2	Dataset Characteristics	2
3	Image Preprocessing Pipeline	2
4	Segmentation Methodologies and Logic	3
5	Performance Evaluation Metrics	4
5.1	Execution Time (ms)	4
5.2	Object Density (%)	4
5.3	Contrast Separation Ratio (CSR)	4
6	Experimental Benchmark Results	6
6.1	Quantitative Analysis Summary Matrix	6
6.2	Key Data Interpretations	7
6.3	Analytical Performance Chart Visualizations	7
6.4	Qualitative Sample Matrix Outputs	8
6.5	Project Repository Verification	9
A	Appendix: Code Implementations	10
A.1	Preprocessing Pipeline Implementation	10
A.2	Image Segmentation Implementation	10

1 Introduction and Project Objectives

Image segmentation serves as a cornerstone of intermediate computer vision systems, acting as the critical bridge between raw low-level pixel grids and high-level semantic object comprehension. By subdividing a digital canvas into visually distinct, uniform regions matching real-world boundaries, segmentation permits targeted downstream processing.

This project designs and executes a rigorous empirical evaluation benchmarking six classic image segmentation paradigms. Rather than relying on computationally heavy, data-dependent deep neural models, this research investigates the granular mathematical capabilities and execution properties of classical spatial, edge, grouping, and topological extraction systems.

2 Dataset Characteristics

To comprehensively evaluate how robustly each traditional approach adapts to varied visual distributions, a structural multi-class testing dataset containing a total of **3,606 unique images** was utilized. The underlying collection is explicitly categorized across 7 distinct object profiles. These classes present a balanced mixture of sharp artificial profiles (man-made geometries with low internal variance) and organic, complex textured subjects exhibiting high pixel variation:

Table 1: Comprehensive Dataset Category and Sample Size Distribution

Class Profile	Structural Group Type	Total Extracted Files
Cars	Artificial / Low Textural Variance	840
Bike	Artificial / Intricate Structural Borders	730
Flowers	Organic / High Chromatic Contrast	420
Dogs	Organic / Non-Rigid Deformable Textures	404
Cats	Organic / Complex Background Confounding	404
Horses	Organic / Variant Spatial Orientations	404
Human	Organic / Highly Variable Silhouettes	404
Total Evaluation Pool —		3,606

3 Image Preprocessing Pipeline

Traditional vision algorithms are highly sensitive to microscopic environmental perturbations, lighting anomalies, and ambient camera sensor noise. To stabilize feature extraction and maintain algorithmic fairness across variant classes, all incoming images are passed through a tightly controlled sequential 7-step digital cleanup optimization pipeline prior to invoking segmentation mechanisms.



Figure 1: Active visualization of the sequential transformations within the preprocessing pipeline.

The functional steps of the preprocessing framework are structurally detailed below:

1. **Spatial Dimensional Standardization:** Inputs are scaled uniformly to a fixed resolution of 256×256 pixels. This standardizes the operational area, ensuring that fixed filter kernels and morphologic radii behave identically regardless of original pixel densities.
2. **Luminance Reduction (Grayscale):** Color data is collapsed into an 8-bit single-channel intensity array. Classical approaches isolate boundaries using structural changes in luminance rather than color spaces.
3. **Impulse Noise Suppression (Median Filter):** A non-linear 5×5 median blur window systematically replaces outlier pixels with neighborhood medians, completely removing salt-and-pepper noise while maintaining crisp object edges.
4. **High-Frequency Smoothing (Gaussian Filter):** A linear low-pass 5×5 Gaussian kernel ($\sigma = 0$) performs a localized weighted average. This removes micro-textures and subtle surface grain that would otherwise cause false edge or gradient triggers.
5. **Contrast Limited Adaptive Histogram Equalization (CLAHE):** Standard histogram transformations over-amplify background noise. CLAHE avoids this by computing equalization locally within a grid of 8×8 tiles and clipping the maximum histogram slope at a limit of 2.0 to enhance local boundary contrast cleanly.
6. **Morphological Opening:** An erosion operation followed immediately by dilation using a flat 5×5 rectangular structuring element cleans up remaining tiny bright artifacts or speckles.
7. **Morphological Closing:** A dilation step followed by erosion using the same 5×5 kernel seals minor internal gaps or holes within the primary target contours, yielding a smooth, stable image layout.

4 Segmentation Methodologies and Logic

Six standard computer vision frameworks representing distinct analytical philosophies are applied directly over the preprocessed single-channel image space:

- **Adaptive Thresholding:** Rather than applying a single global brightness cutoff across the entire canvas, this framework computes a localized threshold dynamically for every pixel based on a Gaussian-weighted mean inside an active 11×11 pixel window minus a constant ($C = 2$). This logic effectively separates objects across datasets plagued by uneven lighting or shadowing.
- **Canny Edge Detection:** An edge-focused extraction engine that maps sudden brightness changes. It calculates a spatial gradient magnitude map and isolates true boundaries using non-maximum suppression alongside dual hysteresis thresholds (scaled tightly around the image's median intensity). While excellent at extracting crisp silhouettes, it outputs independent lines rather than closed solid regions.
- **Otsu's Thresholding:** A global thresholding approach that automatically calculates an optimal brightness cutoff value by analyzing the entire grayscale intensity histogram. It treats the image as a bimodal mixture and loops through all possibilities to find the exact threshold that minimizes intra-class variance. This method works best on images containing high contrast between the object and background.
- **K-Means Clustering:** An unsupervised grouping algorithm that reshapes the intensity matrix into a 1D vector and clusters pixels into discrete groups ($K = 2$) using iterative distance optimization. It excels at parsing distinct color or brightness tiers but does not consider the spatial relationship between pixels.

- **Region Growing:** A region-based extraction scheme that deploys outward from a fixed seed coordinate positioned at the geometric center of the canvas (128, 128). The algorithm loops through neighboring pixels and absorbs them into the foreground mask if their absolute intensity variance stays within a specific tolerance limit (± 15). It performs beautifully when the target object directly dominates the center of the frame.
- **Watershed Segmentation:** A topological approach that interprets the input grayscale intensity map as a geographic landscape, where bright values represent mountain peaks and dark values represent valleys. Using an internal Euclidean distance transform and connected components to establish markers, it floods the terrain from separate basins until the rising water elements meet, cleanly separating touching components.

5 Performance Evaluation Metrics

To evaluate the segmented masks systematically across all 3,606 images without manual human annotations, three distinct unsupervised computational and statistical benchmarks are established:

5.1 Execution Time (ms)

Captures the computational overhead by calculating the precise hardware runtime delta (T_{exec}) from the instant the segmentation function is called to when its output mask is returned.

- **Analytical Importance:** This metric tracks real-time viability. Point-level operations (like Otsu’s and Adaptive thresholding) execute almost instantly by evaluating individual pixel states in isolation. Conversely, clustering and flooding pipelines (like K-Means or Watershed) require iterative data updates or topological tracking, demanding higher computational budgets.

5.2 Object Density (%)

Tracks the percentage (D_{obj}) of the standardized 256×256 workspace that is classified as foreground (white pixels with a value of 255) versus background (black pixels with a value of 0):

$$D_{\text{obj}} = \left(\frac{\sum P_{\text{foreground}}}{\sum P_{\text{total}}} \right) \times 100 \quad (1)$$

- **Analytical Importance:** Object density serves as an excellent indicator of systemic under- or over-segmentation. If an algorithm’s average density tracks extremely close to 0% or 100%, it indicates a failure to extract distinct regions, meaning the model is either treating the entire canvas as empty background or leaking entirely into the surrounding frame.

5.3 Contrast Separation Ratio (CSR)

Quantifies the statistical isolation between the extracted foreground object and its surrounding background space by comparing their intensity profiles:

$$\text{CSR} = \frac{|\mu_{\text{fg}} - \mu_{\text{bg}}|}{\sigma_{\text{fg}} + \sigma_{\text{bg}} + \epsilon} \quad (2)$$

Where μ_{fg} and μ_{bg} denote the mean grayscale intensity values, σ_{fg} and σ_{bg} represent the standard deviations within those isolated zones, and $\epsilon = 10^{-5}$ acts as a smoothing factor to prevent division-by-zero errors.

- **Analytical Importance:** A higher CSR value proves that the algorithm has successfully isolated a region with an intensity distribution distinct from its background. Algorithms that optimize for global variance minimization (like Otsu or K-Means) naturally achieve high CSR values, especially on high-contrast datasets.

6 Experimental Benchmark Results

6.1 Quantitative Analysis Summary Matrix

Table 2 catalogs the global performance averages across all evaluated test parameters, sorted directly by algorithmic grouping to illuminate trends:

Table 2: System Evaluation Summary Matrix Across Image Class Categories

Category	Algorithm	Object Density (%)	Contrast Separation	Execution Time (ms)
cats	Canny_Edge	4.65	0.13	0.52
cars	Canny_Edge	4.66	0.15	0.50
horses	Canny_Edge	4.74	0.24	0.54
dogs	Canny_Edge	5.12	0.13	0.54
human	Canny_Edge	5.20	0.19	0.51
bike	Canny_Edge	5.84	0.12	0.55
flowers	Canny_Edge	6.98	0.11	0.65
bike	Watershed	25.18	0.88	3.86
cars	Watershed	25.33	1.12	3.48
horses	Watershed	26.21	1.25	3.52
cats	Watershed	30.79	1.31	3.58
human	Watershed	30.84	1.35	3.46
dogs	Watershed	31.19	1.24	3.77
flowers	Watershed	32.68	1.24	4.11
flowers	K_Means	36.25	1.43	17.74
bike	K_Means	41.77	1.04	17.03
cats	K_Means	46.40	1.49	17.14
dogs	K_Means	48.77	1.45	17.88
human	K_Means	53.37	1.68	17.16
cars	K_Means	62.30	1.32	16.59
horses	K_Means	62.61	1.44	17.17
flowers	Otsu_Threshold	45.04	1.38	0.10
dogs	Otsu_Threshold	48.65	1.42	0.11
cats	Otsu_Threshold	48.75	1.45	0.10
bike	Otsu_Threshold	50.36	1.06	0.10
human	Otsu_Threshold	52.48	1.54	0.12
cars	Otsu_Threshold	55.62	1.28	0.10
horses	Otsu_Threshold	55.92	1.40	0.12
flowers	Adaptive_Threshold	64.29	0.36	0.37
dogs	Adaptive_Threshold	69.11	0.37	0.41
bike	Adaptive_Threshold	69.62	0.45	0.37
cats	Adaptive_Threshold	70.94	0.36	0.36
human	Adaptive_Threshold	71.15	0.56	0.37
cars	Adaptive_Threshold	71.18	0.44	0.38
horses	Adaptive_Threshold	71.22	0.51	0.38
bike	Region_Growing	96.88	0.40	0.44
flowers	Region_Growing	97.99	0.28	0.52
horses	Region_Growing	98.16	0.63	0.42
cats	Region_Growing	98.20	0.38	0.41
dogs	Region_Growing	98.57	0.39	0.46
cars	Region_Growing	98.61	0.49	0.38
human	Region_Growing	98.61	0.56	0.42

6.2 Key Data Interpretations

- **Computational Latency:** Otsu's thresholding establishes the absolute performance baseline for speed, requiring only ≈ 0.10 ms per execution due to its efficient 1D histogram search logic. Canny Edge and Adaptive structures follow closely under 0.6 ms. Conversely, K-Means Clustering requires the largest computational budget (≈ 17.0 ms), showing the cost of iterative multidimensional centroid optimization steps.
- **Density Profiles:** Region Growing shows heavy boundary leaking ($> 96\%$ output density uniformly across all categories), proving highly sensitive to texture variations. Canny stays minimal ($\approx 5\%$) because it creates fine lines rather than closed areas. Watershed maintains highly stable, realistic regional spatial counts ($\approx 25\% - 32\%$).
- **Contrast Separation:** K-Means and Otsu generate high CSR readings (> 1.30), proving highly effective at cleanly splitting bimodal distributions (e.g., Cars, Human) based on global intensity values.

6.3 Analytical Performance Chart Visualizations

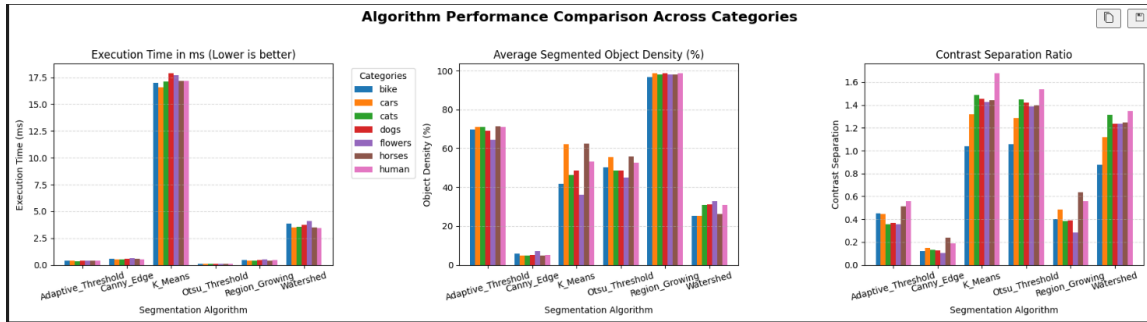
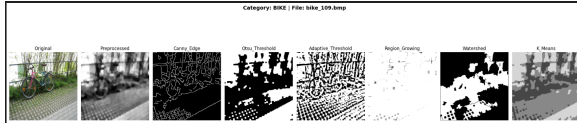


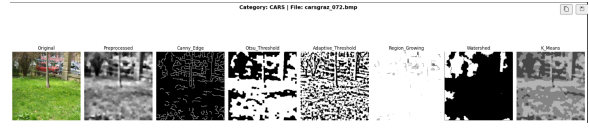
Figure 2: Comparative performance charts mapping execution speed, spatial object density, and contrast separation across target classes.

6.4 Qualitative Sample Matrix Outputs

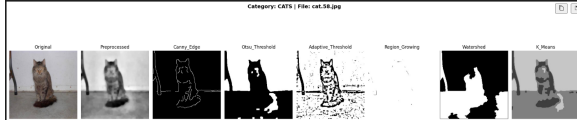
The figures below demonstrate the qualitative visual behavior of each algorithm across the seven object classes:



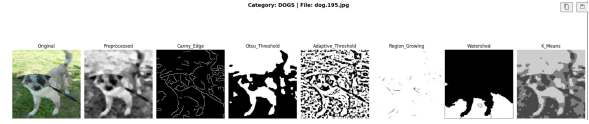
(a) Bike Segmentation Row



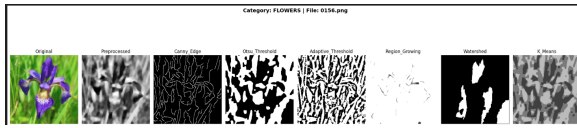
(b) Cars Segmentation Row



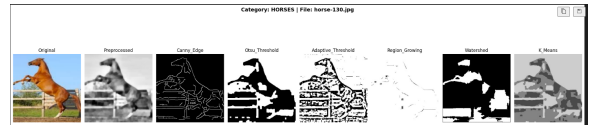
(c) Cats Segmentation Row



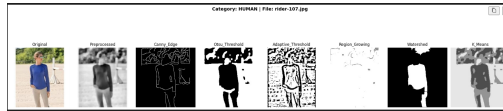
(d) Dogs Segmentation Row



(e) Flowers Segmentation Row



(f) Horses Segmentation Row



(g) Humans Segmentation Row

Figure 3: Comprehensive cross-categorical qualitative matrix comparisons.

6.5 Project Repository Verification

For direct replication and a full code review of the pipeline framework, the open-source hosting layout can be reached through the reference marker below:



Figure 4: QR Code link targeting the official project repository hosting layout.

A Appendix: Code Implementations

A.1 Preprocessing Pipeline Implementation

The following Python implementation details the sequential 7-step optimization chain designed to clean and normalize raw images before segmentation is executed:

Listing 1: Sequential Preprocessing Pipeline Script

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Pick the very first image found to inspect the preprocessing chain
test_img_path = image_paths[0]
img = cv2.imread(test_img_path)

% 1. Resize
img_resized = cv2.resize(img, (256, 256))
% 2. Grayscale
gray = cv2.cvtColor(img_resized, cv2.COLOR_BGR2GRAY)
% 3. Median Blur (Removes salt-and-pepper noise)
median = cv2.medianBlur(gray, 5)
% 4. Gaussian Blur (Smooths high-frequency noise)
gaussian = cv2.GaussianBlur(median, (5, 5), 0)
% 5. CLAHE (Enhances local contrast)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
clahe_img = clahe.apply(gaussian)
% 6. Morphological Opening (Removes small bright artifacts)
kernel = np.ones((5,5), np.uint8)
opening = cv2.morphologyEx(clahe_img, cv2.MORPH_OPEN, kernel)
% 7. Morphological Closing (Fills small dark holes)
preprocessing_final = cv2.morphologyEx(opening, cv2.MORPH_CLOSE, kernel)

# Plotting every step for your report
steps = [cv2.cvtColor(img_resized, cv2.COLOR_BGR2RGB), gray, median, gaussian, c
titles = ['1.- Original-(Resized)', '2.- Grayscale', '3.- Median-Blur', '4.- Gaussian-Blur', '5.- CLAHE', '6.- Morphological Opening', '7.- Morphological Closing']
```

A.2 Image Segmentation Implementation

The following segment defines the six traditional core image segmentation mathematical frameworks evaluated over the dataset:

Listing 2: Traditional Segmentation Algorithms Implementation Suite

```
# =====
# ALGORITHM DEFINITIONS
# =====

def run_canny(preprocessed, _):
    return cv2.Canny(preprocessed, 80, 180)

def run_otsu(preprocessed, _):
    _, res = cv2.threshold(preprocessed, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    return res
```

```

def run_adaptive(preprocessed, _):
    return cv2.adaptiveThreshold(preprocessed, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN,

def run_region_growing(preprocessed, _):
    h, w = preprocessed.shape
    mask = np.zeros((h+2, w+2), np.uint8)
    res = preprocessed.copy()
    cv2.floodFill(res, mask, (w//2, h//2), 255, 15, 15)
    return res

def run_kmeans(preprocessed, _):
    pixel_vals = preprocessed.reshape((-1, 1)).astype(np.float32)
    criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0)
    _, labels, centers = cv2.kmeans(pixel_vals, 3, None, criteria, 1, cv2.KMEANS_
    centers = np.uint8(centers)
    return centers[labels.flatten()].reshape((preprocessed.shape))

def run_watershed(preprocessed, img_resized):
    _, thresh = cv2.threshold(preprocessed, 0, 255, cv2.THRESH_BINARY_INV + cv2.
    sure_bg = cv2.dilate(thresh, np.ones((3,3), np.uint8), iterations=2)
    dist_transform = cv2.distanceTransform(thresh, cv2.DIST_L2, 5)
    _, sure_fg = cv2.threshold(dist_transform, 0.4 * dist_transform.max(), 255,
    unknown = cv2.subtract(sure_bg, np.uint8(sure_fg))
    _, markers = cv2.connectedComponents(np.uint8(sure_fg))
    markers = markers + 1
    markers[unknown == 255] = 0

    img_watershed = img_resized.copy()
    cv2.watershed(img_watershed, markers)

    # Create a binary mask of the segmented objects for accurate metrics
    binary_mask = np.zeros_like(preprocessed, dtype=np.uint8)
    binary_mask[markers > 1] = 255
    return binary_mask

methods = {
    'Canny_Edge': run_canny,
    'Otsu_Threshold': run_otsu,
    'Adaptive_Threshold': run_adaptive,
    'Region_Growing': run_region_growing,
    'K_Means': run_kmeans,
    'Watershed': run_watershed
}

```